

8/9/26

Context Free Grammars

CS340
Lecture

(CFG)

CFGs allow us to define a set recursively.
Let us look at an example.

Example 1:

We may define a class of simple arithmetic expressions as follows.

$$E \rightarrow id$$

// id stands for an
// identifier

$$E \rightarrow E + E$$

$$E \rightarrow E * E$$

$$E \rightarrow (E)$$

Rule 1 ($E \rightarrow id$) may be thought of as the base case in this defn. while other rules may be seen as specifying other forms of expressions ~~recursively~~.
~~(Reading a rule from left to right is~~ inductively.

For example $E \rightarrow E * E$ says that if there are two expressions then their product is also an expression.

E is a syntactic category ~~to~~ defining all simple expressions.

Here is an example of how to derive a member of this category.

$$E \Rightarrow E + E \Rightarrow E + E * E \Rightarrow^* id + id * id$$

More formally this definition may be presented as the following CFG.

$$(\{E\}, \{id, +, *\}, \{E \rightarrow id, E \rightarrow E + E, E \rightarrow E * E, E \rightarrow (E)\}, E)$$

set of variables or syntactic categories

set of terminals

production rules

start symbol

Defn A CFG G is a four tuple (V, T, P, S)

where V is a set of variables, T is a set of terminals $V \cap T = \emptyset$. P is a set of rules of the form $A \rightarrow \alpha$, where $A \in V$ and $\alpha \in \{V \cup T\}^*$

$S \in V$ is called start symbol.

Let $\alpha, \beta \in \{V \cup T\}^*$, we define ~~as~~ ~~not~~ derivability relation $\Rightarrow_G$ $\alpha \Rightarrow_G \beta$ as follows.

Defn Let ~~$\alpha = \alpha_1 \alpha_2 \dots \alpha_n$~~

$$\alpha = \alpha_1 A \alpha_2$$

If $A \rightarrow \alpha_3 \in P$ (production rules of G) then $\alpha \Rightarrow_G \alpha_1 \alpha_3 \alpha_2$.

Relation $\Rightarrow_G^*$ is reflexive transitive closure of $\Rightarrow_G$. (we omit subscript G if grammar G is clear from the context).

We saw earlier $E \Rightarrow^* E + E * E$

and $E + E * E \Rightarrow^* id + id * id$

Def Let G be a CFG. The language $L(G)$ associated with G is defined as

$$L(G) = \{ w \in T^* \mid S \Rightarrow_G^* w \}$$

($L(G)$ is also called the language generated by G .)

Example 2:

Consider Grammar $G = (\{S\}, \{0, 1\}, \{S \rightarrow 01, S \rightarrow 0S1\}, S)$

What is $L(G)$?

Let us examine α 's d.f. $S \Rightarrow^* \alpha$

$$S \Rightarrow 0S1 \Rightarrow 00S11 \Rightarrow \dots \Rightarrow 0^n S 1^n$$

applying second rule

general form of α derivable from S

To obtain $w \in \{0, 1\}^*$, we need to apply first rule once in the end. Therefore,

$$L(G) = \{ 0^n 1^n \mid n \geq 1 \}$$

Defn

A language L is a context free language (CFL) iff there is a ~~grammar G s.t.~~ CFG G s.t. $L = L(G)$.

We saw above that $\{0^n 1 \mid n \geq 1\}$ is a CFL.
We know that this is not a regular language.
So, there are CFL's which are not regular.

Not all CFLs are as easy to analyse as the above examples.

Example 3: Consider $G = (V, T, P, S)$,

where $V = \{S, A, B\}$ $T = \{a, b\}$

P is

$$\begin{aligned} S &\rightarrow aB \mid bA \mid \epsilon \\ A &\rightarrow aS \mid bS a A \mid a \\ B &\rightarrow aS b B \mid bS \mid b \end{aligned}$$

Can you figure out $L(G)$?

Try to associate a language with each of the three variables. Then prove your guess by induction on $|w|$, for $w \in \{a, b\}^*$.

Example 4:

Consider ~~the~~ a CFG

$$G' = (\{S\}, \{a, b\}, P, \{S\}),$$

where P is $S \rightarrow \epsilon \mid aSbS \mid bSaS$

What is $L(G')$?

In fact, $L(G) = L(G')$, can you show that?

We have seen that there are CFLs which are not regular. Can we show that every regular language is a CFL? There are many ways to show that. Here we suggest one, based on ^{induction on} regular expressions.

Exercise 1: Let R, S be regular expressions over alphabet Σ then show that if $L(R), L(S)$ are CFLs so are $L(R.S)$ and $L(R^*)$. ~~For other~~ ~~ways~~ Alternatively, one can just show that CFLs are closed under ' $\cup$ ', ' $\cdot$ ' and ' $*$ ' operations on languages. Conclude that ~~every~~ every regular expression defines a CFL.

Combining with example 2, we get that: class of CFLs is strictly larger ~~than~~ than the class of regular languages.

Context Free Grammars get their name, because they were first investigated by linguists in order to formalize parts of natural languages. 'Context free' prefix in grammar refers to the fact only a single non-terminal appears on the left of each production rule. CFGs have many uses in computer science. For example, in specifying parts of programming language syntax, and in compiler technology.

We saw an example of derivation $E \Rightarrow^* id + id * id$ earlier.

$$\begin{aligned} E &\Rightarrow E + E \Rightarrow E + E * E \Rightarrow id + E * E \\ &\quad \Downarrow \\ &\quad id + id * E \Rightarrow id + id * id \end{aligned}$$

We could also derive the same string (expression) by applying the rules in a different order, for example,

$$\begin{aligned} E &\Rightarrow E + E \Rightarrow id + E \Rightarrow id + E * E \Rightarrow id + E * id \\ &\quad \Downarrow \\ &\quad id + id * id \end{aligned}$$

Def A derivation $S \Rightarrow^* \alpha$ is said to be leftmost derivation, if at any step ⁱⁿ it the leftmost non-terminal is replaced using a production rule. A rightmost derivation is defined analogously. These are also written as $S \xRightarrow{lm}^* \alpha$ or $S \xRightarrow{rm}^* \alpha$, respectively.

Remark: Above defn is appropriate when $\alpha \in T^*$. For α which may have some non-terminals, we need to say that non-terminals are expanded in left to right order (but not all of them may be expanded).

A leftmost derivation for $id + id * id$ in the ~~arithmetic~~ arithmetic expression's grammar is:

$$E \Rightarrow E + E \Rightarrow id + E \Rightarrow id + E * E \Rightarrow id + id * E$$

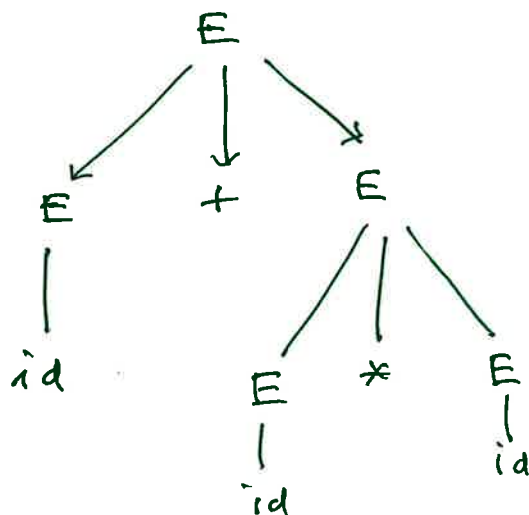
$\downarrow$
 $id + id * id$

As another example, following is a leftmost derivation $S \Rightarrow^* aababbab$ in the grammar of example-4 (recall the production rules: $S \rightarrow \epsilon | aSbS | bSaS$)

$$\begin{aligned}
 S &\Rightarrow a \underline{S} b S \xRightarrow{(S \rightarrow \epsilon \text{ used})} a a \underline{S} b S b S \xRightarrow{(S \rightarrow \epsilon \text{ used})} a a b \underline{S} b S \\
 &\quad \swarrow \xRightarrow{S \rightarrow \epsilon} a a b a \underline{S} b S b S \quad \searrow \xRightarrow{S \rightarrow \epsilon} a a b a b \underline{S} b S \\
 &\quad \downarrow \xRightarrow{S \rightarrow \epsilon} a a b a b b \underline{S} \quad \downarrow \xRightarrow{S \rightarrow \epsilon} a a b a b b a \underline{S} \\
 &\quad \quad \quad \downarrow \xRightarrow{S \rightarrow \epsilon} a a b a b b a b
 \end{aligned}$$

An alternative way to present a derivation is by a tree, called parse tree.

For example, ~~leftmost~~ derivation of $E \Rightarrow^* id + id * id$ may be presented as



Parse tree representation is more abstract than linear representation of derivation. For example, all ~~derivations~~ linear derivations of $E \Rightarrow^* id + id * id$ shown above lead to the same parse tree shown above. This is because, order in which subtrees in a parse tree are expanded is not part of the parse tree.

To be completely formal, we define a ~~parse~~ parse tree as follows.

Def: A tree T ~~labeled~~ with nodes labeled by symbols from set $V \cup T$ is a parse tree if for every internal node u with children $u_1 \dots u_n$, from left to right, ' $l(u) \rightarrow l(u_1) l(u_2) \dots l(u_n)$ ' is a production rule in G .

(Here $l(u)$ is the label of vertex u in T)

Yield of a parse tree is ^{the} content of all leaves read from left to right.

(and root is labeled by the start symbol)
If yield of T is α then we also say that T is a parse tree for α in G .

Exercise 2:

Let T be a parse tree ~~is~~ for CFG G , with the root vertex labeled by a non-terminal A and yield of T being α . Then show that $A \xRightarrow{*}_G \alpha$.

(Hint: use induction on height of ~~is~~ T)

We know that each derivation $\equiv A \xRightarrow{*}_G \alpha$ defines a unique parse tree T with root labeled A and yield α . Also, we have seen that several derivations $\equiv A \xRightarrow{*}_G \alpha$ may map to ~~the~~ the same parse tree. However, a parse tree corresponds to a unique leftmost (rightmost) derivation.

Lemma: Let T be a parse tree for G , with root labeled as A and $\text{yield}(T) = \alpha$. ~~Then~~ Then there is a unique leftmost (rightmost) derivation $A \xRightarrow{*}_{\text{lm}} \alpha$ of G which gives rise to T .

Proof: Let $A \rightarrow x_1 \dots x_n$ be the production rule applied at the root node of T .

$\text{Yield}(T) = \alpha_1 \dots \alpha_n$ where α_i is the yield of the subtree rooted at x_i .

(if x_i is a terminal then $\alpha_i = x_i$)

The ~~is~~ leftmost derivation

$$A \Rightarrow x_1 \dots x_n \xRightarrow{*}_{\text{lm}} \alpha_1 x_2 \dots x_n \xRightarrow{*}_{\text{lm}} \alpha_1 \alpha_2 x_3 \dots x_n \dots \Rightarrow \alpha_1 \alpha_2 \dots \alpha_n$$

Can be defined by induction on height of the tree.

□